

# SQL Query and Result Summary

## 1. Order and Sales Analysis

### ♦ *Total Orders and Revenue*

```
sql
CopyEdit
SELECT
    COUNT(*) AS total_orders,
    SUM(order_amount) AS total_revenue
FROM customer_orders;
```

#### **Insight:**

There are 14,668 total orders, with a significant total revenue (amount hidden in the dataset).

### ♦ *Order Status Distribution*

```
sql
CopyEdit
SELECT
    order_status,
    COUNT(*) AS order_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS percentage,
    SUM(order_amount) AS total_amount
FROM customer_orders
GROUP BY order_status
ORDER BY order_count DESC;
```

#### **Insight:**

- Orders are almost evenly distributed across statuses:
  - Shipped: 4,897 (33.39%)
  - Delivered: 4,874 (33.24%)
  - Pending: 4,897 (33.39%)

#### ♦ *Monthly Revenue Trend*

sql

CopyEdit

```
SELECT
    YEAR(order_date) AS year,
    MONTH(order_date) AS month,
    COUNT(*) AS order_count,
    SUM(order_amount) AS total_sales
FROM customer_orders
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY year, month;
```

#### **Insight:**

Order volumes per month are consistently between 210-240 with no clear growth trend.

#### ♦ *Highest Order Amount*

sql

CopyEdit

```
SELECT TOP 1 WITH TIES
    order_id,
    customer_id,
    order_amount
FROM customer_orders
ORDER BY order_amount DESC;
```

#### **Insight:**

Highest order amount is \$499.90.

### ◇ *Average Order Value by Status*

```
sql
CopyEdit
SELECT
    order_status,
    AVG(order_amount) AS avg_order_value,
    MIN(order_amount) AS min_order_value,
    MAX(order_amount) AS max_order_value
FROM customer_orders
GROUP BY order_status;
```

#### **Insight:**

Order values are fairly consistent across statuses.

## **2. Customer Analysis**

### ◇ *Total and Unique Customers*

```
sql
CopyEdit
SELECT COUNT(DISTINCT customer_id) AS total_customers
FROM customer_orders;
```

#### **Insight:**

7,334 unique customers.

### ◇ *Repeat Customers*

```
sql
CopyEdit
```

```

SELECT
    COUNT(*) AS repeat_customer_count
FROM (
    SELECT customer_id
    FROM customer_orders
    GROUP BY customer_id
    HAVING COUNT(*) > 1
) AS repeat_customers;

```

**Insight:**

4,402 repeat customers (≈60% retention rate).

◊ *Customer Segmentation*

sql

CopyEdit

```

WITH cust_orders AS (
    SELECT customer_id, COUNT(order_id) AS order_count
    FROM customer_orders
    GROUP BY customer_id
),
segmented AS (
    SELECT
        CASE
            WHEN order_count = 1 THEN 'One-time'
            WHEN order_count BETWEEN 2 AND 3 THEN 'Occasional'
            ELSE 'Frequent'
        END AS segment
    FROM cust_orders
)
SELECT segment, COUNT(*) AS customer_count
FROM segmented
GROUP BY segment;

```

**Insight:**

- One-time: 2,932
- Occasional: 3,586
- Frequent: 816

#### ♦ *Avg Orders per Customer*

sql

CopyEdit

```
SELECT
    AVG(order_count) AS avg_orders_per_customer
FROM (
    SELECT customer_id, COUNT(order_id) AS order_count
    FROM customer_orders
    GROUP BY customer_id
) AS customer_order_counts;
```

#### **Insight:**

Average orders per customer  $\approx 2$ .

#### ♦ *Avg Time Between Orders*

sql

CopyEdit

```
WITH customer_orders_ranked AS (
    SELECT
        customer_id,
        order_date,
        LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY
order_date) AS next_order_date
    FROM customer_orders
)
SELECT
    AVG(DATEDIFF(day, order_date, next_order_date)) AS
Avg_days_between_orders
```

```
FROM customer_orders_ranked
WHERE next_order_date IS NOT NULL;
```

**Insight:**

Average time between orders = 479 days.

### 3. Payment Status Analysis

#### ◇ *Payment Summary*

```
sql
CopyEdit
SELECT
    payment_status,
    COUNT(*) AS transaction_count,
    SUM(payment_amount) AS total_amount
FROM payments
GROUP BY payment_status;
```

**Insight:**

~33% each for completed, pending, and failed.

#### ◇ *Payment Failure Rate by Method*

```
sql
CopyEdit
SELECT
    payment_method,
    SUM(CASE WHEN payment_status = 'failed' THEN 1 ELSE 0 END) AS
failed_txn,
    COUNT(*) AS total_txn,
    ROUND(100.0 * SUM(CASE WHEN payment_status = 'failed' THEN 1 ELSE
0 END) / COUNT(*), 2) AS failure_rate_pct
FROM payments
```

GROUP BY payment\_method;

**Insight:**

All payment methods have ≈33% failure rate.

## 4. Order Details Report

### ◇ *Missing Payments*

```
sql
CopyEdit
SELECT COUNT(*)
FROM customer_orders o
LEFT JOIN payments p ON o.order_id = p.order_id
WHERE p.order_id IS NULL;
```

**Insight:**

5,500 orders have missing payment data.

### ◇ *Delivered But Failed Payments*

```
sql
CopyEdit
SELECT COUNT(*)
FROM customer_orders o
JOIN payments p ON o.order_id = p.order_id
WHERE o.order_status = 'delivered' AND p.payment_status = 'failed';
```

**Insight:**

1,650 delivered orders had failed payments.

### ♦ *Full Order-Payment Report*

```
sql
CopyEdit
SELECT
    co.order_id,
    co.customer_id,
    co.order_date,
    co.order_amount,
    co.order_status,
    p.payment_status,
    p.payment_amount,
    CASE
        WHEN p.payment_status = 'completed' AND p.payment_amount >=
co.order_amount THEN 'Fully Paid'
        WHEN p.payment_status = 'completed' AND p.payment_amount <
co.order_amount THEN 'Partially Paid'
        WHEN p.payment_status = 'failed' THEN 'Payment Failed'
        WHEN p.payment_id IS NULL THEN 'No Payment Record'
        ELSE 'Payment Pending'
    END AS payment_summary
FROM customer_orders co
LEFT JOIN payments p ON co.order_id = p.order_id
ORDER BY co.order_date DESC;
```

## 5. Customer Retention Cohort Analysis

```
sql
CopyEdit
WITH cohort AS (
    SELECT
        customer_id,
        DATEFROMPARTS(YEAR(MIN(order_date)), MONTH(MIN(order_date))),
1) AS cohort_month
    FROM customer_orders
    GROUP BY customer_id
),
```



```

orders_by_month AS (
    SELECT
        customer_id,
        DATEFROMPARTS(YEAR(order_date), MONTH(order_date), 1) AS
order_month
    FROM customer_orders
),
cohort_analysis AS (
    SELECT
        c.customer_id,
        c.cohort_month,
        o.order_month
    FROM cohort c
    JOIN orders_by_month o ON c.customer_id = o.customer_id
)
SELECT
    cohort_month,
    order_month,
    COUNT(DISTINCT customer_id) AS retained_customers
FROM cohort_analysis
GROUP BY cohort_month, order_month
ORDER BY cohort_month, order_month;

```